

웹 게임 운영 인프라 프로젝트

KT CLOUD PERSONAL PROJECT

유준형

CONTENTS

1 프로젝트 개요

2 핵심 아키텍처

3 주요 설계사항

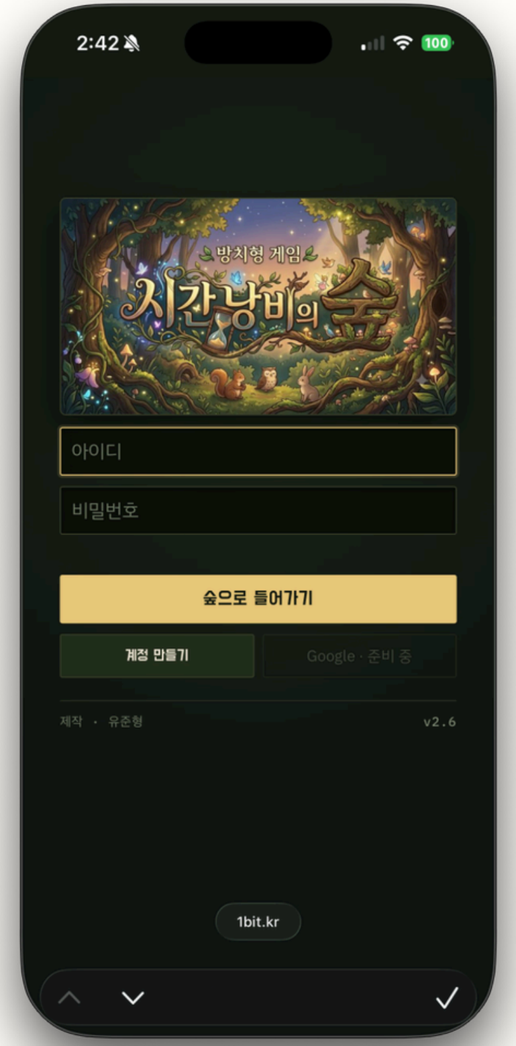
4 기술 포인트

5 트러블슈팅

6 프로젝트를 마치며

OVERVIEW

프로젝트 개요



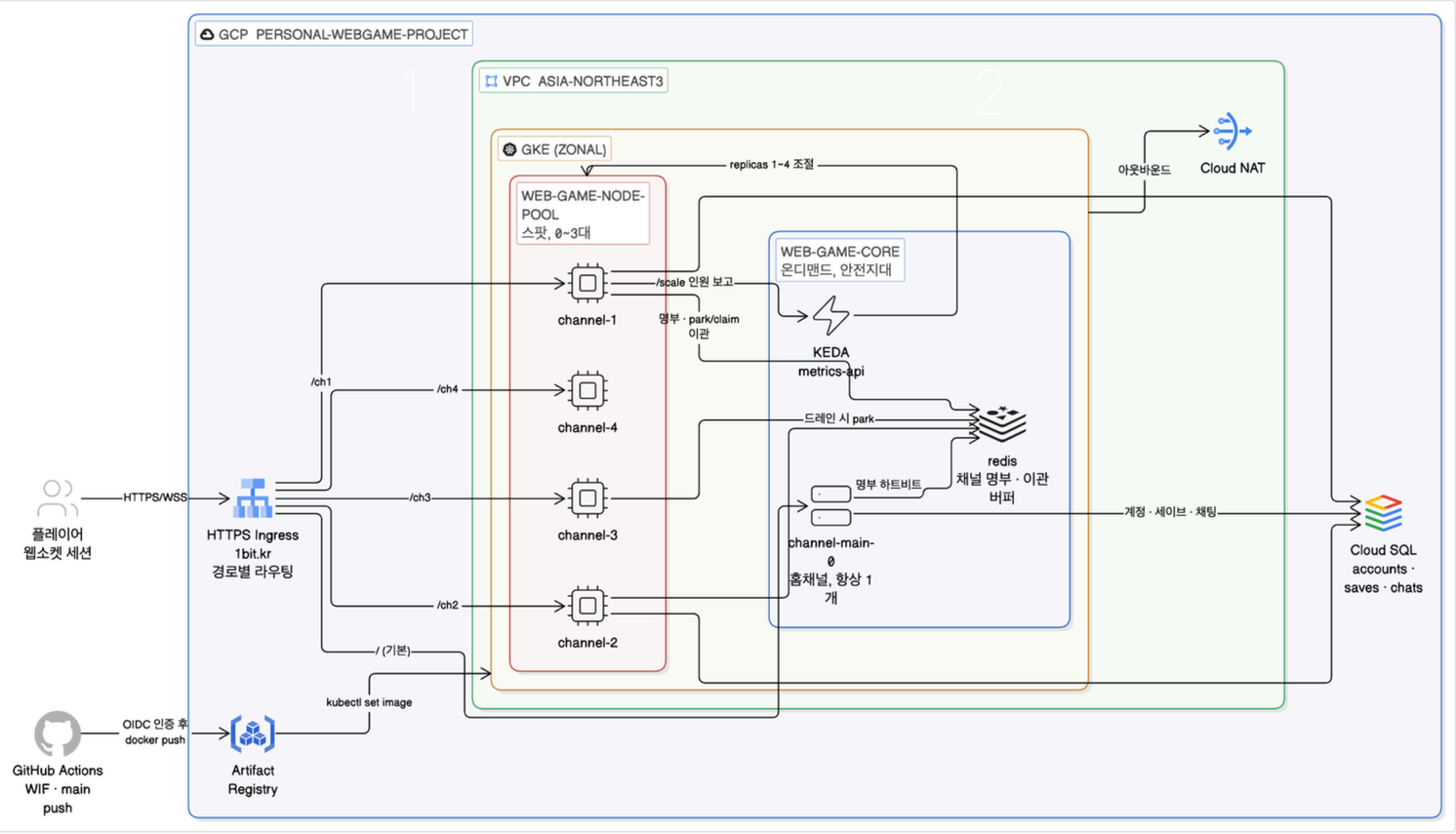
저는 평소에도 게임사의 인프라 운영에 관심이 있었습니다. 이에 AI를 활용해 PC/모바일 통합 웹 게임을 직접 설계하고, 이를 클라우드 환경에서 운영하는 프로젝트를 계획하고 진행했습니다.

원래는 팀 프로젝트 후보안으로 제안했으나, 게임 개발 쪽에 리소스가 매몰된다는 이유로 후보에서 제외되었지만, 아쉬운 마음에 해당 아이템을 개인 프로젝트로 채택하였습니다.

실제로 제가 해왔던 게임들과 최대한 비슷하게 구성해보고자 했고, 채널 하나를 파드 하나로 구현해 정원이 차면 다음 채널로 자동 안내되도록 만들었습니다. 계정·세이브·채팅 같은 사용자 정보는 DB(Cloud SQL)에 저장해, 서버가 바뀌거나 채널을 옮겨도 데이터가 유지되게 했습니다.

ARCHITECTURE

핵심 아키텍처



channel-main-0(홈 채널, 항상 고정) channel-1~4(spot instance, KEDA를 통해 replicas 조절)

온디맨드(core)와 스팟(node-pool) 노드 풀을 분리해 스팟 선점에도 홈 채널은 절대 안 죽게 구성했다.

GitHub Actions + WIF로 main push만 하면 빌드부터 GKE 롤아웃까지 자동으로 끝난다.

KEY DECISIONS

주요 설계 사항

[테라폼을 통한 프로비저닝 on Google Cloud Platform]

지난 해커톤 진행 당시, GCP를 짧게 사용해 아쉬웠던 점, 크레딧이 많이 남은 점을 감안하여 GCP에 테라폼을 사용하여 프로비저닝했습니다.

[노드풀을 온디맨드+스팟으로 분리]

비용 절감을 위해, autoscaling 노드들은 모두 spot 인스턴스로 설정하고, 항상 유지되어야 할 메인 채널은 온디맨드로 구성했습니다.

[Redis는 버퍼로, 진짜 데이터는 Cloud SQL로]

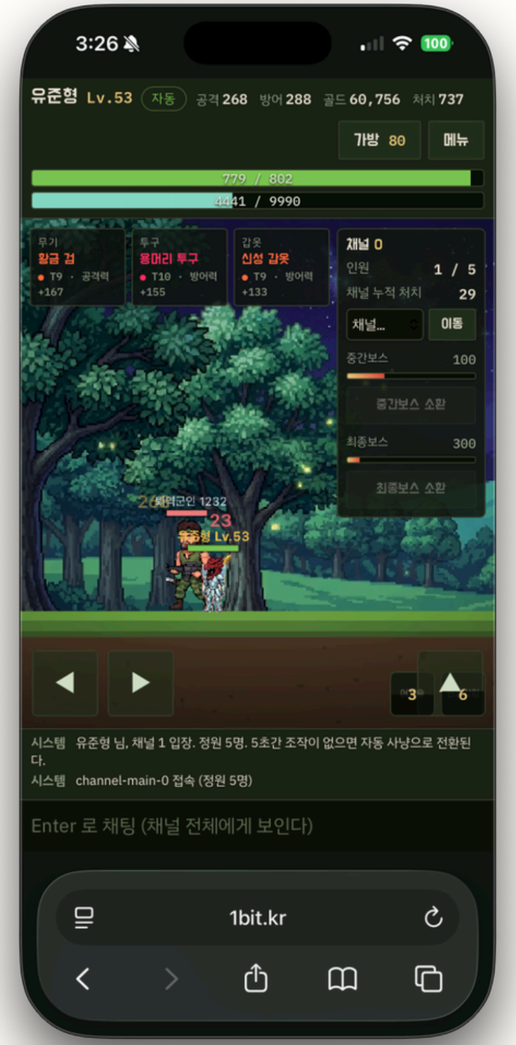
채널 명부는 TTL 하트비트로 자동 소멸시키고, 세션 이관은 park/claim으로만 처리 - 영구 저장은 전부 Cloud SQL에 위임했습니다.

[채널이 가득차면, 다른 채널로]

채널 정원 5명이 차면 클라이언트가 채널 명부(/channels)를 보고 다음 채널로 자동 접속을 시도합니다.

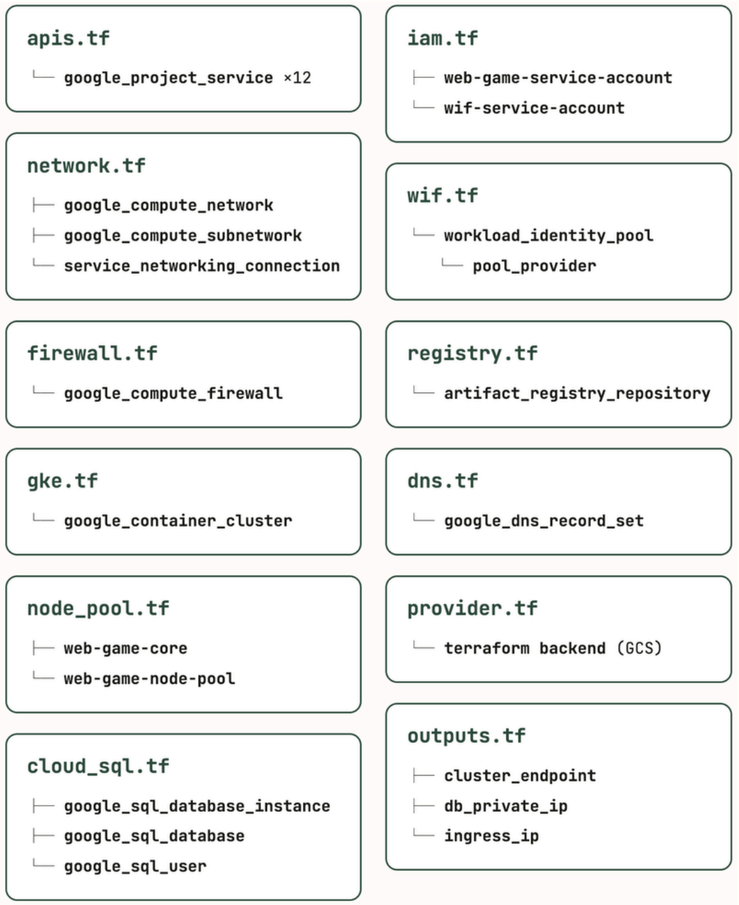
[Github Action을 통한 cicd 자동화]

게임 코드, k8s 매니페스트, GitHub Actions 워크플로 중 하나라도 바뀌면, push 한 번으로 빌드부터 GKE 배포까지 자동으로 끝납니다.



SKILL POINTS

기술 포인트



[테라폼을 통한 프로비저닝 on Google Cloud Platform]

provider.tf에서 Terraform 상태 파일을 GCS 버킷에 저장하도록 설정했고, 나머지 .tf 파일들은 리소스 단위로 나눠 작성했습니다.

GCP는 AWS와 달리 사용하려는 기능마다 API를 직접 '활성화'해줘야 해서, apis.tf에서 컴퓨트·GKE·Cloud SQL 등 필요한 API 12개를 먼저 켜는 것부터 시작했습니다.
방화벽도 AWS의 보안그룹처럼 인스턴스 하나하나에 붙이는 방식이 아니라, google_compute_firewall로 네트워크 전체에 규칙을 거는 방식이라 이 차이에 맞춰 설정했습니다.

Cloud SQL 연결 방식도 달랐습니다. AWS RDS는 VPC 서브넷에 바로 배치하면 되지만, GCP는 Private Service Access로 별도 피어링을 맺어야 Private IP로 연결할 수 있어 network.tf에 이 설정을 추가로 작성했습니다.

SKILL POINTS

기술 포인트

```
jason@MacBookAir web_game_project % k get pods -o wide
NAME                    READY   STATUS    RESTARTS   AGE     IP          NODE                                                  NOMINATED NODE
channel-1               1/1     Running   0          2m32s   10.4.0.2    gke-web-game-cluster-web-game-node-po-4d8b0fcf-w54w   <none>
channel-main-0          1/1     Running   0          3h9m    10.4.2.54   gke-web-game-cluster-web-game-core-1c35c484-cx56      <none>
redis-54fc7c5ddf-vqmb8  1/1     Running   0          27h     10.4.2.2    gke-web-game-cluster-web-game-core-1c35c484-cx56      <none>
jason@MacBookAir web_game_project %
```

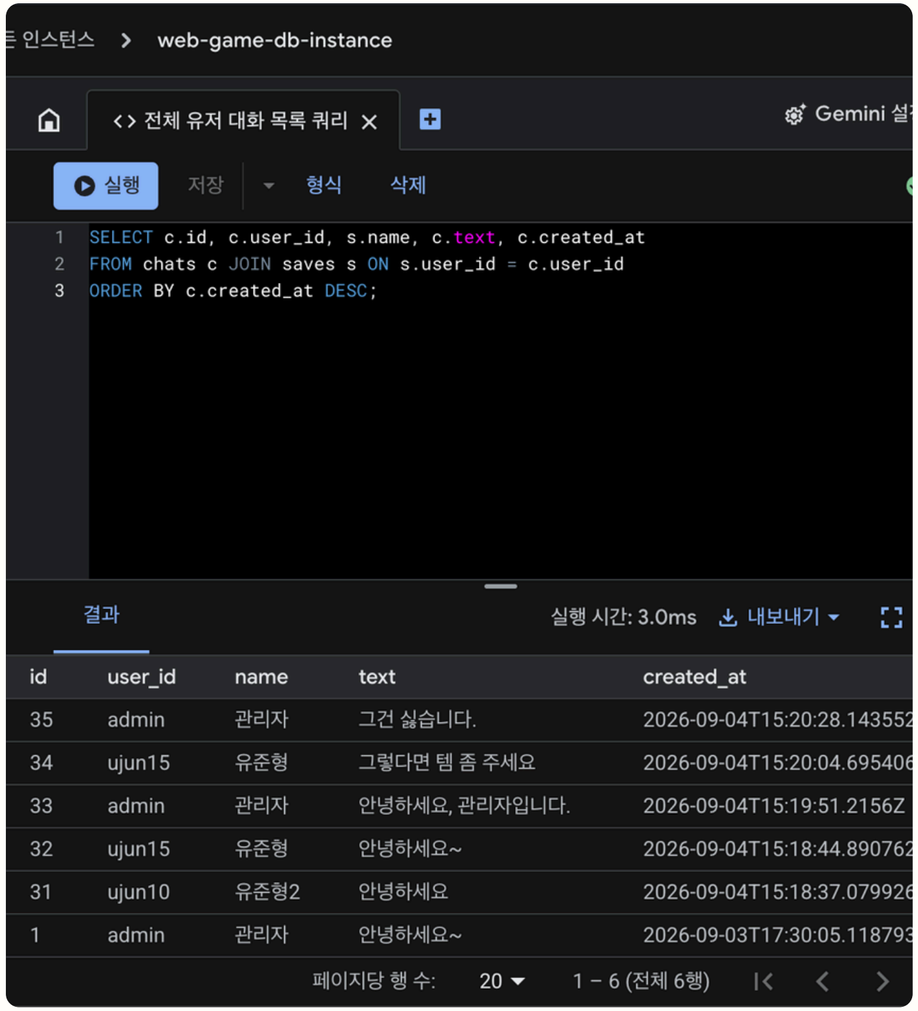
[노드풀을 온디맨드+스팟으로 분리]

스팟 노드 1개 구성으로 운영할 때, 스팟 인스턴스가 예고 없이 통째로 선점당하면 서비스 전체가 같이 다운되는 문제가 있었습니다. 이를 해결하기 위해 노드풀을 web-game-core(온디맨드)와 web-game-node-pool(스팟) 두 개로 분리했습니다.

redis, KEDA와 홈 채널(channel-main-0)은 온디맨드 core 노드에 고정해 절대 죽지 않는 안전지대로 두었고, sub 채널(channel-1~4)만 스팟 노드에서 값싸게 운영하도록 구성했습니다. 그 결과 스팟 노드가 흔들려도 첫 접속(홈 채널)은 항상 성공하고, sub 채널만 일시적으로 영향을 받는 구조가 되었습니다.

SKILL POINTS

기술 포인트



[Redis는 버퍼로, 진짜 데이터는 Cloud SQL로]

채널 명부와 세션 이관 데이터처럼 짧게 살다 사라지는 데이터만 Redis에 두었습니다. 채널 명부는 5초마다 하트비트를 찍고 TTL 15초로 관리해서, 파드가 죽으면 별도 정리 로직 없이 자동으로 사라지도록 했습니다.

반대로 계정, 캐릭터 세이브, 채팅처럼 영구히 남아야 하는 데이터는 전부 Cloud SQL에 저장했습니다. accounts, saves, chats 세 테이블로 나눠서, 로그인 계정(accounts)과 캐릭터 진행도(saves)를 분리했고, saves가 accounts를, chats가 saves를 FK로 참조하도록 설계했습니다. 계정이 삭제되면 세이브·채팅도 같이 정리됩니다.

이렇게 나눈 이유는 역할이 다르기 때문입니다. Redis는 "지금 이 순간"만 빠르게 조회하면 되는 캐시 용도이고, Cloud SQL은 재부팅되거나 다른 채널로 옮겨도 절대 사라지면 안 되는 진짜 데이터를 위한 곳입니다.

SKILL POINTS

기술 포인트



[채널이 가득차면, 다른 채널로]

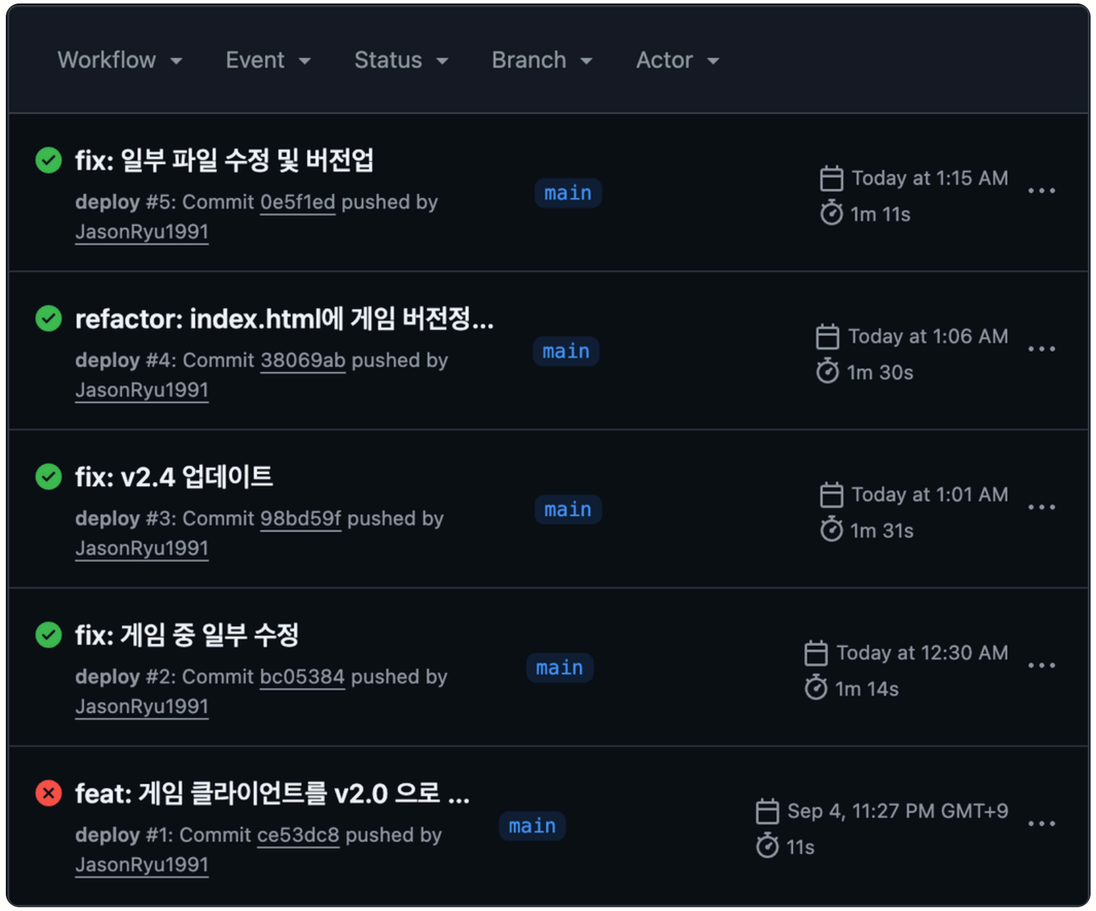
채널 정원은 5명입니다. 정원이 차면 클라이언트가 채널 명부(/channels)를 확인해 다음 채널로 자동으로 접속을 시도합니다.

전체 인원이 늘어나면 KEDA가 sub 채널 파드 수를 조절합니다. 15초마다 전체 인원을 targetValue(3)로 나눈 값을 기준으로 필요한 채널 개수를 산출해 1~4개 사이에서 스케일합니다. 다만 이미 접속해있는 인원을 채널 간에 재배치하지는 않고, 새로 들어오는 접속자만 명부를 보고 덜 찬 채널로 분산됩니다.

이렇게 신규 접속자 분산(클라이언트)과 채널 증설(KEDA)이 같이 동작해야 인원이 몰려도 게임이 끊기지 않습니다.

SKILL POINTS

기술 포인트



[Github Action을 통한 cicd 자동화]

main 브랜치에 field/ 또는 k8s/ 변경분을 push하면 GitHub Actions가 자동으로 실행됩니다.

실행 순서는 field/ 스모크 테스트 → WIF(Workload Identity Federation)로 GCP 인증 → docker build/push → kubectl set image로 두 StatefulSet에 이미지 교체 → rollout status로 배포 완료 확인입니다.

WIF를 써서 GCP 서비스 계정 키 파일을 GitHub에 저장하지 않고, GitHub의 OIDC 토큰으로 인증하도록 구성했습니다.

TROUBLESHOOTING

트러블슈팅

#1
SPOT 인스턴스 선점

이슈 발생
- 클라이언트 수가 적었던 초기에는 메인 채널도 스팟 노드에서 운영 중이었는데, 그 스팟 노드가 선점당하면서 서비스 자체가 중지됐습니다.

해결 방법
- 노드풀을 온디맨드(core)+스팟(spot)으로 분리해 redis·홈채널은 core에 고정했습니다.

#2
Ingress 무반응

이슈 발생
- Ingress를 만들었는데 접속이 하나도 안 됐습니다. 클러스터를 destroy/recreate하면서 ingressClassName이 참조하던 gce 리소스가 사라진 게 원인이었습니다.

해결 방법
- ingressClassName 대신 어노테이션 방식으로 바꿔 리소스 없이도 항상 인식되도록 했습니다.

#3
WIF 소프트 삭제

이슈 발생
- 비용을 아끼려고 인프라를 통째로 destroy했는데, CI/CD 인증 기반인 WIF 풀도 같이 삭제됐습니다. GCP는 삭제해도 30일간 소프트 삭제 상태로 남아 재생성이 막힌다는 걸 그때 알았습니다.

해결 방법
- gcloud CLI로 소프트 삭제된 Workload Identity Provider를 undelete로 복구하고, terraform import로 상태 파일에 다시 편입시켰습니다.

REFLECTION

프로젝트를 마치며



비전공자 수준의 IaC 능력을 가진 저에게는, 부족한 부분을 직접 찾아보고 이해해볼 수 있었던 뜻깊은 프로젝트였습니다.

물론 이 프로젝트를 진행하며 AI의 도움도 받았지만, 최대한 직접 작성해보려 노력했습니다.
그 과정에서 기존에 학습했던 AWS와는 다른 환경의 Terraform 리소스를 찾아 작성해보고, kubeadm으로 노드에 직접 세팅하는 방식이 아니라 Control plane이 분리된 형태의 GKE라는 관리형 클러스터를 세팅해보기도 했습니다.
해커톤 때 팀원분이 세팅하셨던 GitHub Actions도 직접 세팅해봤고, 웹 게임을 직접 인프라에 띄워 지인들에게 접속해보라고 재촉했던 재밌는 프로젝트였습니다.

하지만 한편으로는 아쉬움도 많이 남는 프로젝트였습니다. 계획했던 것보다 시간을 훨씬 적게 쓰게 되어, 웹 게임 쪽은 이동 중에도 원격으로 AI를 활용해 시간 매몰을 줄이려 했지만 그럼에도 시간 분배에는 실패하여, 프로젝트 후반부에는 인프라 영역의 AI 의존도도 높아지게 되었습니다.
그리고 여전히 provider 문서를 매번 다시 찾아봐야 하고, 손에 익지 않은 k8s 명령어들 때문에 아직 많이 부족하다는 생각이 드는 프로젝트였습니다.



THANK YOU FOR YOUR ATTENTION

시간 내어 읽어 주셔서 감사합니다

1bit.kr는 지금도 라이브로 운영 중입니다.
코드와 인프라 구성은 GitHub 레포에서 확인하실 수 있습니다.



https://1bit.kr



https://github.com/JasonRyu1991/web_game_project